

Compiladores

Trabalho Prático

Luís Kennedy Gervásio Turola

Thaís Giovanna Lopes

Tobias Maugus Bueno Cougo

Departamento de Ciências da Computação
Ciências da Computação

Lavras, Julho 2026

Índice

1	Introdução	2
1.1	Objetivo	2
1.2	Etapas do Trabalho	2
2	Analisador Léxico	3
2.1	Descrição Geral da Implementação	3
2.2	Decisões de Projeto	3
2.2.1	Uso de Tabela <i>Hash</i> para Identificadores	3
2.2.2	Separação entre <i>Tokens</i> Simples e Compostos	4
2.2.3	Tratamento Explícito de Comentários	4
2.2.4	Tratamento do Operador de Subtração e Sinal Negativo	5
2.3	Dificuldades Encontradas	5
2.3.1	Implementação da Tabela <i>Hash</i>	5
2.3.2	Tratamento de Comentários Multilinha	5
2.4	Saída do Analisador	6
2.4.1	Arquivo de Entrada	6
2.4.2	Saída Produzida	6
2.4.3	Tabela de Símbolos	8
2.5	Autômatos Finitos Determinísticos (<i>DFAs</i>)	8
2.5.1	Identificadores	8
2.5.2	Números	9
2.6	Análise do Resultado	9
3	Analisador Sintático	10
3.1	Descrição Geral da Implementação	10
3.2	Integração entre o Analisador Léxico e Sintático	11
3.3	Gramática da Linguagem	11
3.4	Decisões de Projeto	14
3.4.1	Uso de Precedência para Resolução de Ambiguidades	14
3.4.2	Mensagens de Erro Sintático	15
3.4.3	Uso de <code>YY_USER_ACTION</code>	15
3.4.4	Declaração de Múltiplos Identificadores	16
3.4.5	Estratégia de Recuperação de Erros	16

3.5	Dificuldades Encontradas	17
3.5.1	Tratamento de Comentários Multilinha com EOF	17
3.5.2	Implementação da Recuperação de Erros	17
3.6	Conjuntos FIRST e FOLLOW	18
3.6.1	Conjuntos FIRST	18
3.6.2	Conjuntos FOLLOW	19
3.7	Estados do Autômato LR(0)	20
3.7.1	Conflito do <i>Dangling Else</i>	20
3.7.2	Conflito entre Subtração e Negação Unária	22
3.8	Saída do Analisador	23
3.8.1	Arquivo de Entrada	23
3.8.2	Saída Produzida	23
3.8.3	Exemplo de Falha Léxica	24
3.9	Conclusão	25
4	Análise Semântica e Geração de Código Intermediário	27
4.1	Descrição Geral da Implementação	27
4.2	Estrutura da Árvore Sintática Abstrata	28
4.2.1	Estrutura dos Nós	28
4.2.2	Tradução Dirigida por Sintaxe	29
4.3	Tabela de Símbolos e Gerenciamento de Escopos	31
4.3.1	Estrutura dos Símbolos	31
4.3.2	Estrutura da Tabela de Símbolos	31
4.3.3	Gerenciamento de Escopos	32
4.3.4	Inserção e Busca de Símbolos	33
4.4	Análise Semântica	35
4.4.1	Visão Geral da Análise Semântica	35
4.4.2	Verificação de Declarações	35
4.4.3	Sistema de Tipos	36
4.4.4	Verificação de Expressões	38
4.4.5	Verificação de Chamadas de Função	39
4.4.6	Relato de Erros Semânticos	40
4.5	Geração de Código Intermediário	41
4.5.1	Visão Geral da Geração de Código	41
4.5.2	Estrutura do Código de Três Endereços	43
4.5.3	Temporários e Rótulos	44
4.5.4	Geração de Código para Expressões	45
4.5.5	Geração de Código para Estruturas de Controle	46
4.5.6	Geração de Código para Funções	48
4.6	Decisões de Projeto	49
4.6.1	Arquitetura em Fases Estritas (Pipeline)	49
4.6.2	Projeto da Árvore Sintática Abstrata	50

4.6.3	Análise Semântica e Sistema de Tipos	51
4.6.4	Geração de Código Intermediário	51
4.7	Dificuldades Encontradas	52
4.7.1	Localização Precisa de Erros Semânticos	52
4.7.2	Implementação da Promoção de Tipos	52
4.7.3	Gerenciamento de Escopos	53
4.8	Saída do Analisador	53
4.8.1	Caso de Sucesso	53
4.8.2	Caso de Erro Semântico	56
4.9	Conclusão	57
Considerações Finais		59

1

Introdução

Etapa Atual: 3 / 3

*Drive do Projeto: **Clique para abrir.** (Deve-se usar o e-mail institucional)*

1.1 Objetivo

O presente trabalho tem como objetivo o desenvolvimento incremental de um compilador para uma linguagem imperativa simplificada, abrangendo desde a análise léxica até a análise semântica e a geração de código intermediário.

Este documento tem como finalidade descrever as decisões de projeto, dificuldades encontradas e resultados obtidos em cada uma das etapas propostas.

1.2 Etapas do Trabalho

O desenvolvimento do compilador foi dividido em três etapas principais:

1. *Etapa 1: Analisador Léxico*

Implementação de um analisador léxico utilizando a ferramenta Flex, responsável por reconhecer tokens da linguagem, identificar erros léxicos e construir a tabela de símbolos.

2. *Etapa 2: Analisador Sintático*

Desenvolvimento do analisador sintático com o uso do Bison, incluindo a definição da gramática da linguagem, tratamento de ambiguidades e validação da estrutura sintática dos programas.

3. *Etapa 3: Análise Semântica e Geração de Código Intermediário*

Implementação da análise semântica, com verificação de tipos e controle de escopos, além da geração de código intermediário no formato de três endereços.

2

Analizador Léxico

2.1 Descrição Geral da Implementação

O analisador léxico foi desenvolvido utilizando a ferramenta *Flex*, sendo responsável por realizar a leitura do código-fonte de entrada e convertê-lo em uma sequência de *tokens*.

A implementação foi baseada na definição de expressões regulares associadas a ações semânticas, permitindo o reconhecimento das principais categorias léxicas da linguagem, tais como: palavras-chave, identificadores, números inteiros e de ponto flutuante, operadores, delimitadores e símbolos de pontuação.

Além da identificação dos *tokens*, o analisador também realiza:

- Impressão do *token*, lexema e posição (linha e coluna);
- Detecção e reporte de erros léxicos;
- Construção e exibição de uma tabela de símbolos;
- Desconsideração de espaços em branco e comentários.

O trecho de código do *Flex* a seguir apresenta um trecho das definições léxicas utilizadas.

1	DIGIT	[0-9]
2	LETTER	[a-zA-Z]
3	ID	{LETTER}({LETTER} {DIGIT})*
4	INT	{DIGIT}+
5	FLOAT	{DIGIT}+ "." {DIGIT}+

2.2 Decisões de Projeto

2.2.1 Uso de Tabela *Hash* para Identificadores

Optou-se pela utilização de uma tabela *hash* para o armazenamento dos identificadores reconhecidos durante a análise léxica.

Essa decisão foi motivada pelos seguintes fatores:

- **Eficiência de acesso:** inserções e buscas com tempo médio constante;
- **Evitar duplicidade:** identificação rápida de lexemas já existentes;
- **Organização:** centralização do gerenciamento de símbolos.

A tabela foi implementada com encadeamento para resolução de colisões, conforme ilustrado abaixo.

```
1 Node* process_id(char *lexema) {
2     unsigned int h = 0;
3
4     for (int i = 0; lexema[i] != '\0'; i++) {
5         h = (h * 31) + lexema[i];
6     }
7
8     h = h % HASH_SIZE;
9
10    Node *current = hash_table[h];
11
12    while (current != NULL) {
13        if (strcmp(current->lexema, lexema) == 0)
14            return current;
15
16        current = current->next;
17    }
18
19    Node *new = (Node*) malloc(sizeof(Node));
20    strcpy(new->lexema, lexema);
21
22    new->next = hash_table[h];
23    hash_table[h] = new;
24
25    return new;
26 }
```

2.2.2 Separação entre *Tokens* Simples e Compostos

Foi adotada uma abordagem distinta para o tratamento de *tokens* simples e compostos. *Tokens* simples, como operadores e delimitadores, são reconhecidos diretamente por regras léxicas.

Por outro lado, *tokens* mais complexos, como identificadores, exigem processamento adicional, incluindo inserção e consulta na tabela de símbolos.

2.2.3 Tratamento Explícito de Comentários

Os comentários foram tratados explicitamente para que não gerassem *tokens* válidos durante a análise léxica.

Comentários de uma linha foram reconhecidos por meio de expressões regulares simples. Já os comentários multilinha foram tratados com o uso de estados no *Flex*, permitindo que o analisador permanecesse em um modo específico até o reconhecimento do delimitador de fechamento.

Essa abordagem foi adotada por proporcionar uma solução mais simples e legível em comparação ao uso de expressões regulares mais complexas, além de facilitar o controle da leitura da entrada durante o processamento dos comentários.

```

1 %x COMMENT
2
3 "/*"      { column += 2; BEGIN(COMMENT); }
4
5 <COMMENT>{
6     "*/"   { column += 2; BEGIN(INITIAL); }
7     \n     { column = 1; }
8     .      { column++; }
9 }

```

2.2.4 Tratamento do Operador de Subtração e Sinal Negativo

O caractere de menos (–) foi tratado exclusivamente como operador aritmético, não sendo incorporado diretamente às definições de números inteiros e de ponto flutuante.

Essa decisão foi adotada para evitar ambiguidades na análise léxica, uma vez que a distinção entre o operador de subtração e o sinal de números negativos depende do contexto sintático em que o símbolo aparece.

Dessa forma, optou-se por delegar essa diferenciação para etapas posteriores do compilador, como a análise semântica.

2.3 Dificuldades Encontradas

2.3.1 Implementação da Tabela *Hash*

A implementação da tabela *hash* apresentou desafios relevantes, principalmente relacionados ao tratamento de colisões e ao gerenciamento de memória.

Foi necessário definir uma função de *hash* eficiente para garantir uma boa distribuição dos identificadores, além de implementar uma estratégia de encadeamento para tratar colisões. Também foi importante garantir que identificadores já existentes não fossem inseridos novamente, evitando redundâncias na tabela.

2.3.2 Tratamento de Comentários Multilinha

O tratamento de comentários multilinha representou uma das principais dificuldades da implementação, especialmente no que diz respeito à manutenção correta da posição no arquivo de entrada.

Embora o reconhecimento dos delimitadores de início e fim não tenha apresentado grande complexidade, surgiram desafios em cenários onde comentários aparecem intercalados com código. Nesses casos, foi necessário garantir que apenas os caracteres relevantes fossem considerados na análise, sem que o conteúdo dos comentários interferisse na contagem de colunas.

Essa questão impacta diretamente a precisão das informações de posição (linha e coluna), utilizadas tanto na exibição dos *tokens* quanto na geração de mensagens de erro.

2.4 Saída do Analisador

Durante a execução, o analisador imprime, para cada *token* reconhecido, seu tipo, lexema e posição no arquivo de entrada. Além disso, ao final da análise, é exibida a tabela de símbolos construída.

2.4.1 Arquivo de Entrada

O exemplo a seguir apresenta um arquivo de teste contendo diferentes construções da linguagem, incluindo declarações, expressões, operadores, comentários e casos de erro.

```

1  /* Teste de comentários
2  de múltiplas linhas */
3  int contador = -10;
4  float preco = 45.90;
5  preco;
6  //Comentário de uma linha
7  if (!verdade || 7) { @
8      print i/2+8*7;
9      read i>=1<9!=0;
10 }
```

2.4.2 Saída Produzida

A saída gerada pelo analisador léxico para o arquivo apresentado é mostrada a seguir. Para cada *token* reconhecido, são exibidos seu tipo, lexema e posição no arquivo (linha e coluna). Além disso, erros léxicos são reportados com mensagens descritivas.

```

===== BEGINNING OF LEXICAL ANALYSIS =====
```

TOKEN	LEXEMA	LIN	COL
KW_INT	int	3	1
(ID, 0x5f5854aa8520)	contador	3	5
ASSIG	=	3	14
(OP_ARIT , -)	-	3	16

```

(INT      , 10      ) | 10      | 3      | 17
SEMCOL           | ;          | 3      | 19
KW_FLOAT         | float        | 4      | 1
Lexical error: Invalid identifier '2preco' (Line: 4, Column: 7)
ASSIG           | =            | 4      | 14
(FLOAT      , 45.90 ) | 45.90       | 4      | 16
SEMCOL           | ;            | 4      | 21
(ID, 0x5f5854aa85a0 ) | preco        | 5      | 1
SEMCOL           | ;            | 5      | 6
KW_IF            | if           | 7      | 1
(PAREN      , (      ) | (            | 7      | 4
(OP_LOG      , !      ) | !            | 7      | 5
(ID, 0x5f5854aa8620 ) | verdade      | 7      | 6
(OP_LOG      , ||     ) | ||           | 7      | 14
(INT      , 7      ) | 7            | 7      | 17
(PAREN      , )      ) | )            | 7      | 18
(BRACE      , {      ) | {            | 7      | 20
Lexical error: '@' (Line: 7, Column: 22)
KW_PRINT         | print        | 8      | 9
(ID, 0x5f5854aa86a0 ) | i            | 8      | 15
(OP_ARIT      , /      ) | /            | 8      | 16
(INT      , 2      ) | 2            | 8      | 17
(OP_ARIT      , +      ) | +            | 8      | 18
(INT      , 8      ) | 8            | 8      | 19
(OP_ARIT      , *      ) | *            | 8      | 20
(INT      , 7      ) | 7            | 8      | 21
SEMCOL           | ;            | 8      | 22
KW_READ          | read         | 9      | 9
(ID, 0x5f5854aa86a0 ) | i            | 9      | 14
(OP_REL       , >=     ) | >=           | 9      | 15
(INT      , 1      ) | 1            | 9      | 17
(OP_REL       , <      ) | <            | 9      | 18
(INT      , 9      ) | 9            | 9      | 19
(OP_REL       , !=     ) | !=           | 9      | 20
(INT      , 0      ) | 0            | 9      | 22
SEMCOL           | ;            | 9      | 23
(BRACE      , }      ) | }            | 10     | 1

```

===== END OF LEXICAL ANALYSIS =====

2.4.3 Tabela de Símbolos

Ao final da execução, o analisador exibe a tabela de símbolos construída, contendo todos os identificadores únicos encontrados no programa.

SYMBOL TABLE (HASH)	
LEXEMA	ADDRESS
i	0x5f5854aa86a0
contador	0x5f5854aa8520
verdade	0x5f5854aa8620
preco	0x5f5854aa85a0

O campo de endereço apresentado na tabela corresponde à posição de memória associada a cada identificador armazenado. Nesta etapa, essa informação foi utilizada unicamente para fins de verificação, permitindo confirmar que um mesmo identificador não é inserido múltiplas vezes na tabela, o que valida o correto funcionamento da estrutura de *hashing* adotada.

Embora a tabela de símbolos implementada nesta etapa armazene apenas o lexema e seu respectivo endereço, sua estrutura pode ser adaptada para incluir novos atributos, caso necessário. Informações adicionais, como tipo e escopo, poderão ser incorporadas em etapas futuras, de acordo com as demandas das próximas fases do compilador.

2.5 Autômatos Finitos Determinísticos (DFAs)

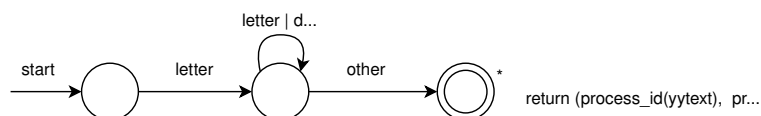
Nesta seção são apresentados os diagramas de autômatos finitos determinísticos (DFAs) correspondentes a duas classes de *tokens* utilizadas no analisador léxico.

2.5.1 Identificadores

A definição utilizada para identificadores é apresentada a seguir:

-
- 1 LETTER [a-zA-Z]
 - 2 DIGIT [0-9]
 - 3 ID LETTER (LETTER | DIGIT)*
-

Essa definição estabelece que um identificador deve iniciar com uma letra, podendo ser seguido por zero ou mais letras ou dígitos.

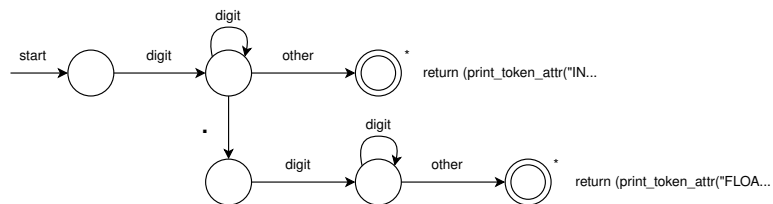


2.5.2 Números

A definição utilizada para números inteiros ou de ponto flutuante é apresentada a seguir:

1	DIGIT	[0-9]
2	INT	{DIGIT}+
3	FLOAT	DIGIT+ "." DIGIT+

Essa definição estabelece que um número inteiro deve possuir pelo menos um dígito e que um número de ponto flutuante deve possuir pelo menos um dígito antes e depois do ponto decimal.



2.6 Análise do Resultado

A partir do exemplo apresentado, é possível observar que o analisador léxico foi capaz de:

- Reconhecer corretamente palavras-chave, identificadores, operadores e literais;
- Ignorar comentários de uma e múltiplas linhas;
- Manter o controle adequado de linhas e colunas;
- Detectar e reportar erros léxicos, como identificadores inválidos e caracteres não reconhecidos;
- Construir corretamente a tabela de símbolos sem duplicação de entradas.

3

Analizador Sintático

3.1 Descrição Geral da Implementação

O analisador sintático foi desenvolvido utilizando a ferramenta *Bison*, sendo responsável por validar a estrutura sintática dos programas escritos na linguagem proposta.

A implementação foi integrada diretamente ao analisador léxico desenvolvido com *Flex*, permitindo que os *tokens* reconhecidos fossem consumidos pelo *parser* em tempo de execução por meio da função `yylex()`.

A gramática implementada suporta:

- Declarações de variáveis com ou sem inicialização;
- Declarações e chamadas de funções;
- Estruturas condicionais `if` e `if-else`;
- Estruturas de repetição `while`;
- Blocos compostos delimitados por chaves;
- Comandos de entrada (`read`) e saída (`print`);
- Comandos de retorno (`return`);
- Expressões aritméticas, relacionais e lógicas;
- Operadores unários de negação lógica e aritmética;
- Comentários de uma e múltiplas linhas.

Além do reconhecimento sintático, o analisador também realiza:

- Detecção de erros sintáticos;
- Impressão de mensagens de erro com linha e coluna;
- Integração com a tabela de símbolos construída pelo analisador léxico;
- Controle de ambiguidades da gramática.

3.2 Integração entre o Analisador Léxico e Sintático

O analisador sintático foi integrado diretamente ao analisador léxico desenvolvido na etapa anterior. Dessa forma, o *parser* consome os *tokens* produzidos pelo *Flex* em tempo de execução por meio da função `yylex()`.

Além do reconhecimento dos *tokens*, o analisador léxico também ficou responsável por atualizar as informações de posição utilizadas pelo *Bison*. Para isso, foi utilizado o contador automático de linhas disponibilizado pelo *Flex* por meio da variável `yylineno`, juntamente com um controle manual de colunas realizado através da variável global `column`.

A atualização dessas informações foi automatizada utilizando a macro `YY_USER_ACTION`, permitindo que cada *token* reconhecido armazenasse:

- linha inicial;
- coluna inicial.

Essas informações passaram a ser utilizadas pelo *Bison* através da estrutura `yyval`, possibilitando a geração de mensagens de erro sintático contendo linha e coluna precisas.

A integração entre *Flex* e *Bison* também possibilitou a reutilização da tabela de símbolos implementada anteriormente, mantendo o armazenamento dos identificadores reconhecidos durante a análise.

3.3 Gramática da Linguagem

A seguir é apresentada a gramática implementada no analisador sintático, descrita em formato BNF simplificado.

```
1 program
2     ::= declaration_list
3
4 declaration_list
5     ::= declaration_list declaration
6        | declaration
7
8 declaration
9     ::= variable_declaration
10        | function_declaration
11
12 variable_declaration
13     ::= type_specifier variable_list ";"
14
15 variable_list
16     ::= variable_list "," variable
17        | variable
18
```

```
19 variable
20     ::= ID
21     | ID "=" expression
22
23 type_specifier
24     ::= "int"
25     | "float"
26
27 function_declaration
28     ::= type_specifier ID "(" parameters ")" compound_stmt
29
30 parameters
31     ::= parameter_list
32     |
33
34 parameter_list
35     ::= parameter_list "," parameter
36     | parameter
37
38 parameter
39     ::= type_specifier ID
40
41 compound_stmt
42     ::= "{" local_declarations statement_list "}"
43
44 local_declarations
45     ::= local_declarations variable_declaration
46     |
47
48 statement_list
49     ::= statement_list statement
50     |
51
52 statement
53     ::= expression_stmt
54     | compound_stmt
55     | selection_stmt
56     | iteration_stmt
57     | io_stmt
58     | return_stmt
59     | ";"
60
61 expression_stmt
62     ::= expression ";"
63
64 expression
65     ::= ID "=" expression
66     | logical_or_expression
67
68 selection_stmt
69     ::= "if" "(" expression ")" statement
70     | "if" "(" expression ")" statement "else" statement
```

```
71
72 iteration_stmt
73     ::= "while" "(" expression ")" statement
74
75 io_stmt
76     ::= "print" "(" expression ")" ";"
77     | "read" "(" ID ")" ";"
78
79 return_stmt
80     ::= "return" expression ";"
81
82 logical_or_expression
83     ::= logical_or_expression "||" logical_and_expression
84     | logical_and_expression
85
86 logical_and_expression
87     ::= logical_and_expression "&&" equality_expression
88     | equality_expression
89
90 equality_expression
91     ::= equality_expression "==" relational_expression
92     | equality_expression "!=" relational_expression
93     | relational_expression
94
95 relational_expression
96     ::= relational_expression "<" additive_expression
97     | relational_expression ">" additive_expression
98     | relational_expression "<=" additive_expression
99     | relational_expression ">=" additive_expression
100    | additive_expression
101
102 additive_expression
103     ::= additive_expression "+" term
104     | additive_expression "-" term
105     | term
106
107 term
108     ::= term "*" factor
109     | term "/" factor
110     | factor
111
112 factor
113     ::= "(" expression ")"
114     | ID
115     | INTNUM
116     | FLOATNUM
117     | function_call
118     | "!" factor
119     | "-" factor
120
121 function_call
122     ::= ID "(" arguments ")"
```

```
123
124 arguments
125     ::= argument_list
126     |
127
128 argument_list
129     ::= argument_list "," expression
130     | expression
```

3.4 Decisões de Projeto

3.4.1 Uso de Precedência para Resolução de Ambiguidades

Durante o desenvolvimento da gramática, os principais conflitos encontrados estiveram relacionados ao problema do *dangling else* e à ambiguidade entre o operador de subtração e o operador de negação unária.

Ao invés de reestruturar completamente a gramática utilizando produções auxiliares, optou-se por resolver esses conflitos utilizando precedência e associatividade fornecidas pelo próprio *Bison*.

```
1 %nonassoc LOWER_THAN_ELSE
2 %nonassoc KW_ELSE
3
4 %left OR
5 %left AND
6 %left EQ NE
7 %left LT GT LE GE
8
9 %left PLUS MINUS
10 %left MULT DIV
11
12 %right NOT
13 %right UMINUS
14 %right ASSIG
```

Essa abordagem permitiu:

- resolver o conflito do *dangling else*;
- diferenciar corretamente subtração binária e negação unária;
- manter a gramática mais simples e legível;
- evitar a criação de produções auxiliares adicionais;
- eliminar conflitos sem aumentar significativamente a complexidade do *parser*.

Além disso, a utilização de precedência tornou a implementação mais simples para a manutenção do fluxo do projeto, considerando que a próxima etapa também será desenvolvida utilizando *Bison*.

3.4.2 Mensagens de Erro Sintático

O projeto exigia a detecção e impressão de erros sintáticos contendo informações claras sobre linha e coluna.

Dessa forma, optou-se por produzir mensagens de erro inspiradas no comportamento de compiladores amplamente utilizados, como *gcc* e *javac*, tornando as mensagens mais compreensíveis e próximas das encontradas em compiladores reais.

Para isso, foi utilizada a diretiva:

```
1 %define parse.error detailed
```

Essa configuração permite que o *Bison* informe:

- *token* inesperado;
- *tokens* esperados pela gramática.

As informações produzidas passaram a ser interpretadas pela função `yyerror()`, responsável por converter mensagens internas do *Bison* em mensagens mais amigáveis ao usuário.

Além disso, também foi utilizada a diretiva:

```
1 %locations
```

Essa funcionalidade disponibiliza a estrutura `yyvalloc`, contendo as posições associadas aos *tokens* reconhecidos pelo *lexer*.

Com isso, as mensagens de erro passaram a indicar:

- símbolo esperado;
- *token* encontrado;
- linha;
- coluna.

3.4.3 Uso de YY_USER_ACTION

Foi utilizada a macro:

```
1 #define YY_USER_ACTION \
2     yyloc.first_line = yylineno; \
3     yyloc.last_line = yylineno; \
4     yyloc.first_column = column; \
5     yyloc.last_column = column + yyleng - 1;
```

Essa macro é executada automaticamente pelo *Flex* sempre que um *token* é reconhecido.

Sua utilização permitiu centralizar a atualização das informações de posição dos *tokens* em um único ponto do *lexer*, evitando repetição de código em todas as regras léxicas.

Além disso, essa abordagem garantiu integração direta com o mecanismo de localização do *Bison*, permitindo que o *parser* tivesse acesso automático às posições associadas a cada *token* consumido durante a análise.

3.4.4 Declaração de Múltiplos Identificadores

A gramática foi projetada para permitir declarações contendo múltiplos identificadores do mesmo tipo.

```
1 int x, y, z;
```

Essa decisão foi adotada para aproximar a linguagem da sintaxe tradicional utilizada em linguagens como C.

3.4.5 Estratégia de Recuperação de Erros

Após a implementação da gramática principal, foi adicionada uma estratégia de recuperação de erros utilizando o símbolo especial *error* disponibilizado pelo *Bison*.

O objetivo dessa abordagem foi permitir que determinados erros sintáticos fossem tratados localmente, evitando o encerramento imediato da análise sempre que possível.

Para isso, foram inseridas produções específicas em pontos estratégicos da gramática, incluindo:

- declarações de variáveis;
- cabeçalhos de funções;
- estruturas condicionais;
- estruturas de repetição;
- comandos de entrada e saída;
- expressões delimitadas por parênteses;
- listas de comandos.

Além disso, foi criada a produção auxiliar:

```
1 recovery_stmt  
2 : error SEMCOL { yyerrok; }  
3 | error RBRACE { yyerrok; }
```

Essa produção permite sincronizar novamente o analisador ao encontrar delimitadores naturalmente presentes na linguagem, como ponto e vírgula ou fechamento de bloco.

Embora a recuperação não elimine todos os possíveis efeitos cascata decorrentes de um erro sintático, ela tornou o analisador mais robusto e permitiu a continuação da análise em diversos cenários sem introduzir conflitos adicionais relevantes na gramática.

3.5 Dificuldades Encontradas

3.5.1 Tratamento de Comentários Multilinha com EOF

Uma das principais dificuldades encontradas esteve relacionada ao tratamento de comentários multilinha não finalizados antes do fim do arquivo.

Inicialmente, o analisador léxico entrava corretamente no estado responsável pelo processamento dos comentários, porém não existia tratamento explícito para o fim de arquivo dentro desse estado. Como consequência, comentários não fechados encerravam silenciosamente a análise sem gerar erro léxico.

Após a adição de uma regra específica para captura de *EOF*, surgiu uma segunda dificuldade: o *parser* passou a produzir erros sintáticos adicionais ao receber um fim de arquivo inesperado.

Para resolver esse problema, foi criada uma flag global indicando a ocorrência de erro léxico fatal. Dessa forma, a função `yyerror()` passou a ignorar mensagens sintáticas geradas após falhas léxicas críticas.

Além disso, também foi necessário armazenar explicitamente a linha e a coluna de início do comentário, permitindo que a mensagem de erro apontasse corretamente onde o comentário não fechado começou.

3.5.2 Implementação da Recuperação de Erros

Outra dificuldade relevante esteve relacionada à implementação de mecanismos de recuperação de erros sintáticos utilizando o símbolo especial `error` do *Bison*.

Durante os primeiros experimentos, a inserção indiscriminada de produções contendo `error` provocou conflitos adicionais na gramática e comportamentos inconsistentes durante o processo de sincronização.

Foi necessário analisar cuidadosamente quais pontos da gramática poderiam atuar como marcos de recuperação sem comprometer o funcionamento do analisador.

Após diversos testes, optou-se por utilizar a recuperação apenas em construções específicas da linguagem, como declarações, comandos delimitados por ponto e vírgula e estruturas delimitadas por chaves ou parênteses.

Essa abordagem permitiu melhorar a robustez do analisador, mantendo a gramática estável e evitando o surgimento de conflitos desnecessários.

3.6 Conjuntos FIRST e FOLLOW

Nesta seção, são apresentados os conjuntos *FIRST* e *FOLLOW* dos principais não-terminais relacionados às expressões da gramática.

3.6.1 Conjuntos FIRST

Não-terminal	FIRST
program	{ KW_INT, KW_FLOAT }
declaration_list	{ KW_INT, KW_FLOAT }
declaration	{ KW_INT, KW_FLOAT }
variable_declaration	{ KW_INT, KW_FLOAT }
variable_list	{ ID }
variable	{ ID }
type_specifier	{ KW_INT, KW_FLOAT }
function_declaration	{ KW_INT, KW_FLOAT }
parameters	{ KW_INT, KW_FLOAT, ϵ }
parameter_list	{ KW_INT, KW_FLOAT }
parameter	{ KW_INT, KW_FLOAT }
compound_stmt	{ LBRACE }
local_declarations	{ KW_INT, KW_FLOAT, ϵ }
statement_list	{ KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, SEMCOL, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, ϵ }
statement	{ KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, SEMCOL, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
expression_stmt	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
selection_stmt	{ KW_IF }
iteration_stmt	{ KW_WHILE }
io_stmt	{ KW_PRINT, KW_READ }
return_stmt	{ KW_RETURN }
expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
logical_or_expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
logical_and_expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
equality_expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
relational_expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
additive_expression	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
term	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
factor	{ LPAREN, ID, INTNUM, FLOATNUM, NOT, MINUS }
function_call	{ ID }
arguments	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, ϵ }

argument_list	{ ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS }
---------------	--

3.6.2 Conjuntos FOLLOW

Não-terminal	FOLLOW
program	{ \$ }
declaration_list	{ KW_INT, KW_FLOAT, \$ }
declaration	{ KW_INT, KW_FLOAT, \$ }
variable_declaration	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
variable_list	{ SEMCOL }
variable	{ SEMCOL, COMMA }
type_specifier	{ ID }
function_declaration	{ KW_INT, KW_FLOAT, \$ }
parameters	{ RPAREN }
parameter_list	{ RPAREN, COMMA }
parameter	{ RPAREN, COMMA }
compound_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
local_declarations	{ KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL }
statement_list	{ RBRACE }
statement	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
expression_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
selection_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
iteration_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
io_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }

return_stmt	{ KW_INT, KW_FLOAT, KW_IF, KW_WHILE, KW_PRINT, KW_READ, KW_RETURN, LBRACE, RBRACE, KW_ELSE, ID, LPAREN, INTNUM, FLOATNUM, NOT, MINUS, SEMCOL, \$ }
expression	{ SEMCOL, COMMA, RPAREN }
logical_or_expression	{ SEMCOL, COMMA, RPAREN, OR }
logical_and_expression	{ SEMCOL, COMMA, RPAREN, OR, AND }
equality_expression	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE }
relational_expression	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE, LT, GT, LE, GE }
additive_expression	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE, LT, GT, LE, GE, PLUS, MINUS }
term	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE, LT, GT, LE, GE, PLUS, MINUS, MULT, DIV }
factor	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE, LT, GT, LE, GE, PLUS, MINUS, MULT, DIV }
function_call	{ SEMCOL, COMMA, RPAREN, OR, AND, EQ, NE, LT, GT, LE, GE, PLUS, MINUS, MULT, DIV }
arguments	{ RPAREN }
argument_list	{ RPAREN, COMMA }

3.7 Estados do Autômato LR(0)

Nesta seção, são apresentados os principais estados do autômato LR(0) relacionados às ambiguidades encontradas durante o desenvolvimento da gramática.

Os conflitos apresentados estão diretamente associados às ambiguidades discutidas anteriormente na Seção de Decisões de Projeto, sendo posteriormente resolvidos por meio do uso de precedência e associatividade no *Bison*.

3.7.1 Conflito do *Dangling Else*

O principal conflito relacionado às estruturas condicionais ocorreu devido ao clássico problema do *dangling else*.

A ambiguidade surge quando o *parser* encontra uma estrutura do tipo:

```

1  if (a)
2      if (b)
3          comando1;
4      else
5          comando2;
```

Nesse caso, ao encontrar o *token* `else`, o *parser* precisa decidir entre:

- reduzir imediatamente a produção correspondente ao `if` interno, encerrando o reconhecimento da estrutura condicional naquele ponto; ou
- continuar o reconhecimento da produção completa do `if-else`, associando o `else` ao `if` mais interno.

A Figura a seguir apresenta os estados do autômato LR(0) associados a esse conflito.



O conflito ocorre principalmente no Estado 114.

Nesse estado, o *parser* já reconheceu:

```
1 KW_IF LPAREN expression RPAREN statement •
```

mas ainda existe a possibilidade de continuar para:

```
1 KW_IF LPAREN expression RPAREN statement • KW_ELSE statement
```

Assim, ao ler o *token* `KW_ELSE`, o analisador possui duas possibilidades:

- realizar uma redução da produção correspondente ao `if` simples;
- executar *shift* do *token* `else`, continuando para o reconhecimento da produção completa do `if-else`.

No autômato apresentado, esse conflito pode ser observado no Estado 114. Nesse ponto, o *parser* pode tanto reduzir a produção correspondente ao `if` interno quanto continuar o reconhecimento da produção completa do `if-else` ao ler o *token* `KW_ELSE`, levando ao Estado 119.

Esse conflito foi resolvido utilizando precedência e associatividade no *Bison*, por meio das diretivas:

```
1 %nonassoc LOWER_THAN_ELSE
2 %nonassoc KW_ELSE
```

e da produção:

```
1 selection_stmt
2 : KW_IF LPAREN expression RPAREN statement
3   %prec LOWER_THAN_ELSE
4 | KW_IF LPAREN expression RPAREN statement
5   KW_ELSE statement
```

Com isso, o *parser* passa a priorizar o *shift* do *token* `else`, associando-o automaticamente ao `if` mais interno, eliminando o conflito sem necessidade de reestruturar a gramática.

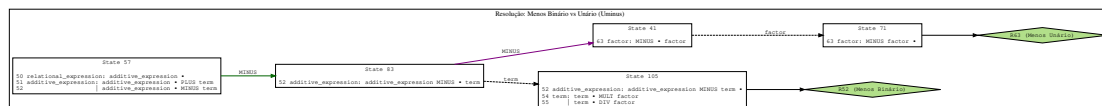
3.7.2 Conflito entre Subtração e Negação Unária

Outro conflito relevante ocorreu devido à ambiguidade entre o operador de subtração binária e o operador de negação unária.

A ambiguidade surge porque ambos utilizam o mesmo símbolo léxico:

```
1 a - b
2 -x
```

A Figura a seguir apresenta os estados do autômato LR(0) associados a esse conflito.



No Estado 57, o *parser* já reconheceu uma expressão aditiva:

```
1 additive_expression: additive_expression • MINUS term
```

permitindo interpretar o símbolo – como operador binário.

Entretanto, ao mesmo tempo, também existe a possibilidade de reconhecer:

```
1 factor: MINUS • factor
```

interpretando o mesmo símbolo como operador unário.

Essa situação gera ambiguidade sobre qual produção deve ser priorizada.

No autômato apresentado, isso pode ser observado pelas transições:

- reconhecimento de MINUS como início de um fator unário;
- reconhecimento de MINUS como continuação de uma expressão aditiva.

Para resolver esse conflito, foi criada uma precedência específica para o operador unário:

```
1 %right UMINUS
```

utilizada na produção:

```
1 | MINUS factor %prec UMINUS
```

Com isso, o *Bison* passa a diferenciar explicitamente os dois usos do operador, eliminando a ambiguidade observada nos estados apresentados pelo autômato LR(0).

3.8 Saída do Analisador

Durante a execução, o analisador imprime os *tokens* reconhecidos pelo *lexer* enquanto o *parser* realiza a validação sintática do programa.

Ao final da execução, o compilador informa se o programa foi aceito ou rejeitado, além de exibir a tabela de símbolos construída.

3.8.1 Arquivo de Entrada

O exemplo a seguir apresenta um programa de teste contendo declarações, chamadas de função, estruturas condicionais, laços de repetição, operadores e comentários.

```

1  int soma(int a, int b) {
2      int resultado;
3      resultado = a + b;
4      return resultado;
5  }
6
7  int main() {
8      int x, y, z;
9
10     read(x);
11     read(y);
12
13     z = soma(x, y);
14
15     if (z > 10) {
16         print(z);
17     }
18     else {
19         z = -10;
20         print(z);
21     }
22
23     while (z > 0) {
24         z = z - 1;
25     }
26
27     return z;
28 }

```

3.8.2 Saída Produzida

```

=====
STARTING LEXICAL AND SYNTACTIC ANALYSIS
=====

```

TOKEN		LEXEMA		LINE		COLUMN
-------	--	--------	--	------	--	--------

```

-----
KW_INT      | int          | 1      | 1
ID          | soma        | 1      | 5
...
ID          | z           | 27     | 12
SEMCOL      | ;           | 27     | 13
RBRACE      | }           | 28     | 1

```

```

=====
SYMBOL TABLE
=====

```

```

LEXEMA      | HASH
-----
a           | 97
b           | 98
x           | 120
y           | 121
z           | 122
soma        | 182
main        | 462
resultado   | 496

```

```

=====
PROGRAM ACCEPTED
=====

```

3.8.3 Exemplo de Falha Léxica

Além dos casos de sucesso, também foram realizados testes contendo erros léxicos, com o objetivo de validar o tratamento de falhas implementado no compilador.

O exemplo a seguir apresenta um comentário multilinha não finalizado antes do fim do arquivo.

```

1  int soma(int a, int b) {
2      int resultado;
3
4      /* Isso é um comentário multilinha
5      para testar algo
6
7      resultado = a + b;
8      return resultado;
9  }

```

Durante a execução, o analisador léxico identifica corretamente que o comentário não foi encerrado e informa a linha e a coluna onde o comentário foi iniciado.

```
=====
STARTING LEXICAL AND SYNTACTIC ANALYSIS
=====

TOKEN          | LEXEMA          | LINE | COLUMN
-----
KW_INT         | int             | 1    | 1
ID             | soma            | 1    | 5
...
KW_INT         | int             | 2    | 5
ID             | resultado       | 2    | 9
SEMCOL         | ;               | 2    | 18
[LEXICAL ERROR] Unterminated comment started at Line 4, Column 5

=====
SYMBOL TABLE
=====
LEXEMA          | HASH
-----
a               | 97
b               | 98
soma            | 182
resultado       | 496
=====

=====
PROGRAM REJECTED
=====
```

Esse teste demonstra que o compilador consegue detectar corretamente comentários multilinha não fechados, interrompendo a análise de forma consistente e evitando a geração de erros sintáticos falsos após o fim do arquivo.

3.9 Conclusão

Apesar das dificuldades encontradas durante o desenvolvimento, foi possível implementar um analisador sintático completo utilizando *Flex* e *Bison*, capaz de reconhecer corretamente programas escritos na linguagem proposta.

O *parser* desenvolvido realiza a validação de declarações, comandos, estruturas condicionais, laços de repetição, chamadas de função e expressões contendo diferentes

níveis de precedência e associatividade.

A integração entre o analisador léxico e o *parser* também permitiu a construção de mensagens de erro contendo linha e coluna precisas, além da reutilização da tabela de símbolos construída durante a análise léxica.

Além disso, decisões de projeto relacionadas ao tratamento de ambiguidades, definição de precedência e associatividade, integração entre *Flex* e *Bison* e implementação de mecanismos de recuperação de erros contribuíram para a construção de um *parser* estável, legível e adequado aos objetivos propostos para esta etapa do compilador.

4

Análise Semântica e Geração de Código Intermediário

4.1 Descrição Geral da Implementação

Nesta etapa foi implementada a análise semântica e a geração de código intermediário para a linguagem desenvolvida nas etapas anteriores.

A arquitetura adotada segue o modelo clássico de compiladores em múltiplas passagens (*multi-pass*), no qual cada fase é executada apenas após a conclusão bem-sucedida da fase anterior. Inicialmente o analisador léxico realiza o reconhecimento dos *tokens*; em seguida o analisador sintático constrói a Árvore Sintática Abstrata (AST); posteriormente a árvore é percorrida para execução das verificações semânticas; e, por fim, o código intermediário é gerado somente quando nenhuma inconsistência semântica é encontrada.

A implementação desenvolvida contempla:

- construção automática da Árvore Sintática Abstrata (AST);
- tabela de símbolos com suporte a escopos aninhados;
- verificação de identificadores declarados e redeclarados;
- verificação de tipos em expressões e atribuições;
- promoção implícita de tipos entre `int` e `float`;
- validação de chamadas de funções e parâmetros;
- geração de código intermediário em formato TAC (*Three-Address Code*);
- geração de temporários e rótulos automaticamente;
- suporte a estruturas condicionais, laços de repetição e chamadas de função.

Além disso, o compilador adota uma estratégia de falha rápida (*fail-fast*), interrompendo imediatamente a compilação quando erros léxicos, sintáticos ou semânticos são encontrados, evitando a produção de código intermediário inconsistente.

4.2 Estrutura da Árvore Sintática Abstrata

4.2.1 Estrutura dos Nós

Como a análise semântica e a geração de código intermediário exigem acesso estruturado ao programa fonte, foi implementada uma Árvore Sintática Abstrata (*Abstract Syntax Tree* – AST).

A AST é construída durante a análise sintática por meio das ações semânticas associadas às produções da gramática. Cada produção cria os nós correspondentes à construção reconhecida, formando uma representação hierárquica do programa que posteriormente será utilizada pelas etapas de análise semântica e geração de código.

Uma decisão importante do projeto foi manter uma separação clara entre as fases do compilador. Dessa forma, durante a análise sintática não são realizadas verificações semânticas nem geração de código intermediário. O analisador sintático tem como única responsabilidade construir a AST completa e armazenar sua raiz para utilização nas etapas posteriores.

A estrutura utilizada para representar os nós da árvore é apresentada a seguir.

```
1 typedef struct ASTNode {
2     ASTNodeType type;
3
4     char *lexema;
5
6     int int_val;
7     float float_val;
8
9     int line;
10    int column;
11
12    char exp_type[20];
13    char temp[20];
14
15    struct ASTNode *child1;
16    struct ASTNode *child2;
17    struct ASTNode *child3;
18    struct ASTNode *child4;
19
20    struct ASTNode *next;
21 } ASTNode;
```

O campo `type` identifica a categoria do nó, permitindo distinguir declarações, comandos, operadores, chamadas de função, literais e demais construções da linguagem.

O atributo `lexema` armazena informações textuais associadas ao nó, como nomes de identificadores, operadores ou literais reconhecidos pelo analisador léxico.

Os campos `int_val` e `float_val` são utilizados para armazenar os valores numéricos de literais inteiros e reais presentes na árvore.

Os atributos `line` e `column` registram a posição original do elemento no código fonte. Essas informações são utilizadas durante a análise semântica para produzir mensagens de erro contendo localização precisa.

O campo `exp_type` é preenchido durante a análise semântica e armazena o tipo inferido da expressão representada pelo nó. Essa informação é utilizada nas verificações de compatibilidade de tipos e também durante a geração do código intermediário.

O atributo `temp` é utilizado na etapa de geração de código intermediário para armazenar o nome do temporário associado ao resultado produzido por determinado nó da árvore.

Os ponteiros `child1` a `child4` representam os filhos diretos do nó, permitindo modelar construções com diferentes quantidades de componentes. A escolha por quatro ponteiros foi motivada pela própria gramática da linguagem implementada, na qual o nó com maior número de componentes é a declaração de função, composta pelo tipo de retorno, identificador, lista de parâmetros e corpo da função. Dessa forma, a estrutura consegue representar diretamente todas as produções da linguagem sem a necessidade de alocação dinâmica de vetores de filhos. Já o ponteiro `next` é utilizado para encadear elementos pertencentes a uma mesma lista, como sequências de comandos, listas de parâmetros ou listas de argumentos.

Após a conclusão da análise sintática, a raiz da AST é armazenada em uma variável global. As fases de análise semântica e geração de código intermediário percorrem essa árvore independentemente, caracterizando uma arquitetura de múltiplas passagens (*multi-pass*) e mantendo as responsabilidades de cada etapa claramente separadas.

4.2.2 Tradução Dirigida por Sintaxe

A construção da Árvore Sintática Abstrata é realizada por meio de Tradução Dirigida por Sintaxe (TDS). Nessa abordagem, cada produção da gramática possui ações semânticas associadas, executadas durante as reduções realizadas pelo analisador sintático. Essas ações são responsáveis por construir os nós da AST e estabelecer as relações entre eles, permitindo que a árvore seja produzida simultaneamente ao reconhecimento da estrutura sintática do programa.

As ações semânticas utilizam a notação tradicional do *Bison*, em que o atributo sintetizado da produção é representado por `$$`, enquanto os atributos dos símbolos do lado direito da regra são acessados por meio de `$1`, `$2`, `$3` e assim sucessivamente. Dessa forma, os nós produzidos por reduções anteriores podem ser reutilizados na construção de estruturas maiores da árvore.

Como exemplo, a Listagem 1 apresenta as ações semânticas responsáveis pela construção dos nós correspondentes às declarações de variáveis.

```
1 variable_declaration
2   : type_specifier variable_list SEMCOL
3   {
4       $$ = create_node(AST_VAR_DECL, $1, $2, NULL, NULL);
```

```

5      };
6
7  variable_list
8      : variable_list COMMA variable
9      {
10         ASTNode *t = $1;
11         while (t->next != NULL)
12             t = t->next;
13
14         t->next = $3;
15         $$ = $1;
16     }
17     | variable
18     {
19         $$ = $1;
20     };
21
22 type_specifier
23     : KW_INT
24     {
25         $$ = create_node(AST_TYPE, NULL, NULL, NULL, NULL);
26         $$->lexema = strdup("int");
27     }
28     | KW_FLOAT
29     {
30         $$ = create_node(AST_TYPE, NULL, NULL, NULL, NULL);
31         $$->lexema = strdup("float");
32     };

```

Listagem 1: Exemplo de ações semânticas para construção da AST durante a análise sintática.

Na produção `type_specifier`, a ação semântica cria um nó do tipo `AST_TYPE` e armazena em seu campo `lexema` o tipo reconhecido (`int` ou `float`). Esse nó é então retornado como atributo sintetizado da produção.

Em seguida, a produção `variable_list` é responsável por construir listas de identificadores. Quando há mais de uma variável declarada, a lista é formada encadeando os nós por meio do ponteiro `next`, permitindo representar declarações contendo múltiplos identificadores utilizando uma única estrutura linear.

Por fim, quando a produção `variable_declaration` é reduzida, os nós produzidos pelas produções anteriores são combinados para formar um único nó `AST_VAR_DECL`. Nesse momento, o nó correspondente ao tipo da declaração é associado ao primeiro filho da árvore, enquanto a lista de variáveis construída anteriormente torna-se o segundo filho. Como resultado, toda a declaração é representada por um único nó da AST, que posteriormente será percorrido pelas etapas de análise semântica e geração de código intermediário.

4.3 Tabela de Símbolos e Gerenciamento de Escopos

4.3.1 Estrutura dos Símbolos

As informações semânticas dos identificadores são armazenadas por meio da estrutura `Symbol`. Cada instância dessa estrutura representa um símbolo declarado no programa, podendo corresponder a variáveis, parâmetros ou funções.

A implementação utilizada no projeto é apresentada a seguir.

```
1 typedef struct Symbol {
2     char name[50];
3     char type[20];
4     char category[20];
5     int num_params;
6     char param_types[10][20];
7     struct Symbol *next;
8 } Symbol;
```

O campo `name` armazena o identificador associado ao símbolo declarado no programa.

O atributo `type` registra o tipo semântico do símbolo, permitindo distinguir declarações dos tipos `int` e `float`.

O campo `category` identifica a categoria do símbolo, diferenciando variáveis, parâmetros e funções durante as verificações semânticas.

O atributo `num_params` é utilizado para armazenar a quantidade de parâmetros formais associados a uma função.

O campo `param_types` mantém os tipos dos parâmetros declarados por cada função, permitindo posteriormente validar chamadas de função quanto à quantidade e compatibilidade dos argumentos fornecidos.

O ponteiro `next` é utilizado para encadear os símbolos pertencentes ao mesmo escopo, formando uma lista ligada armazenada na tabela de símbolos.

As informações armazenadas nessa estrutura são utilizadas posteriormente durante as verificações semânticas e na validação de chamadas de função.

4.3.2 Estrutura da Tabela de Símbolos

Os símbolos declarados durante a compilação são organizados por meio da estrutura `SymbolTable`, responsável por representar um escopo da linguagem.

A implementação utilizada no projeto é apresentada a seguir.

```
1 typedef struct SymbolTable {
2     int table_id;
3     int scope_level;
4     Symbol *symbols;
```

```
5     struct SymbolTable *parent;
6 } SymbolTable;
```

O campo `table_id` identifica unicamente cada tabela criada durante a compilação, facilitando a depuração e a visualização dos escopos construídos.

O atributo `scope_level` armazena a profundidade do escopo representado pela tabela. O escopo global é definido no nível zero, enquanto escopos internos criados por funções ou blocos compostos recebem níveis progressivamente maiores.

O campo `symbols` aponta para o início da lista encadeada de símbolos pertencentes ao escopo atual.

O ponteiro `parent` estabelece a relação hierárquica entre escopos aninhados, permitindo que uma tabela tenha acesso ao escopo imediatamente externo.

Essa organização forma uma árvore de escopos encadeada por ponteiros para os pais, possibilitando que o compilador realize buscas respeitando as regras de visibilidade da linguagem.

4.3.3 Gerenciamento de Escopos

Para suportar escopos aninhados, o compilador mantém um ponteiro global denominado `current_table`, responsável por indicar a tabela de símbolos correspondente ao escopo atualmente ativo durante a análise semântica.

A criação e remoção de escopos são realizadas pelas funções apresentadas a seguir.

```
1 void open_scope() {
2     SymbolTable *new_table = (SymbolTable*)malloc(sizeof(SymbolTable));
3     new_table->symbols = NULL;
4     new_table->parent = current_table;
5     new_table->table_id = total_tables_created;
6     if (current_table) {
7         new_table->scope_level = current_table->scope_level + 1;
8     } else {
9         new_table->scope_level = 0;
10    }
11
12    current_table = new_table;
13    all_tables[total_tables_created] = new_table; // Guarda no histórico
14    total_tables_created++;
15 }
16
17 void close_scope() {
18     if (!current_table) return;
19     // Volta para o pai, mantendo a tabela no histórico
20     current_table = current_table->parent;
21 }
22
```

Sempre que um novo bloco ou função introduz um escopo próprio, para criar uma nova tabela de símbolos é utilizada a função `open_scope()`. A nova tabela passa a apontar para o escopo imediatamente externo por meio do campo `parent`, estabelecendo a hierarquia entre os diferentes níveis de escopo.

O nível de profundidade é armazenado no atributo `scope_level`. O escopo global recebe o nível zero, enquanto cada novo escopo interno recebe um nível igual ao do seu pai acrescido de uma unidade.

Ao término de um bloco, a função `close_scope()` restaura o escopo anterior simplesmente retornando o ponteiro `current_table` para o escopo pai. Dessa forma, os identificadores declarados localmente deixam de estar visíveis para as próximas verificações semânticas.

Essa estratégia implementa escopo estático baseado em encadeamento de tabelas, permitindo que variáveis locais ocultem temporariamente declarações existentes em escopos externos, respeitando as regras tradicionais de visibilidade encontradas em linguagens como C.

Além da tabela corrente, o compilador mantém um histórico de todas as tabelas criadas no vetor `all_tables`. Essa estrutura é utilizada para preservar informações dos escopos construídos durante a compilação, facilitando inspeções e depuração da análise semântica.

4.3.4 Inserção e Busca de Símbolos

Após a definição das estruturas responsáveis pelo armazenamento dos símbolos e pelo gerenciamento dos escopos, foi necessário implementar mecanismos para inserção e consulta de identificadores durante a análise semântica.

A inserção de novos símbolos é realizada pela função `add_symbol()`, apresentada a seguir.

```

1  int add_symbol(const char *name, const char *type, const char *category) {
2      if (!name || !current_table) return 0;
3      // Verifica se a variável já existe no escopo atual
4      Symbol *s = current_table->symbols;
5      while (s) {
6          if (strcmp(s->name, name) == 0) return 0; // Erro de redeclaração
7          s = s->next;
8      }
9
10     // Cria e insere o símbolo no início da lista da tabela atual
11     Symbol *ns = (Symbol*)malloc(sizeof(Symbol));
12     strcpy(ns->name, name);
13     strcpy(ns->type, type);
14     strcpy(ns->category, category);
15     ns->num_params = 0;
16
17     ns->next = current_table->symbols;
18     current_table->symbols = ns;

```

```
19
20     return 1;
21 }
```

Antes de inserir um novo identificador, a função percorre todos os símbolos armazenados na tabela apontada por `current_table`, verificando se já existe um símbolo com o mesmo nome no escopo atual.

Caso um identificador com o mesmo nome seja encontrado, a inserção é cancelada e a função retorna zero, caracterizando um erro de redeclaração.

Quando não existe conflito, um novo registro da estrutura `Symbol` é alocado dinamicamente e inserido no início da lista encadeada de símbolos da tabela corrente.

Essa estratégia garante que dois identificadores não possam ser declarados mais de uma vez dentro do mesmo escopo, ao mesmo tempo em que permite o sombreamento (*shadowing*) de símbolos declarados em escopos externos.

Já a consulta de identificadores é realizada pela função `lookup_symbol()`, apresentada a seguir.

```
1 Symbol* lookup_symbol(const char *name) {
2     if (!name) return NULL;
3
4     SymbolTable *table = current_table;
5     while (table != NULL) {
6         Symbol *s = table->symbols;
7         while (s) {
8             if (strcmp(s->name, name) == 0) return s; // Encontrou
9             s = s->next;
10        }
11        // Se não achou neste escopo, sobe para a tabela pai
12        table = table->parent;
13    }
14    return NULL; // Se chegou à tabela Global e não achou, a variável não existe
15 }
```

A função inicia a busca na tabela atual apontada por `current_table`. Caso o identificador não seja encontrado no escopo atual, a busca continua recursivamente pelos escopos ancestrais através do ponteiro `parent`.

O processo termina quando o símbolo é encontrado ou quando a busca alcança o escopo global sem localizar nenhuma declaração compatível.

Essa estratégia implementa exatamente as regras de visibilidade da linguagem, nas quais identificadores declarados em escopos internos possuem prioridade sobre declarações externas de mesmo nome.

Além de permitir a resolução correta de referências a variáveis e funções, o mecanismo de busca também é utilizado para detectar o uso de identificadores não declarados, possibilitando a emissão de erros semânticos contendo a posição exata do problema no código fonte.

4.4 Análise Semântica

4.4.1 Visão Geral da Análise Semântica

A análise semântica é executada após a conclusão da análise sintática e utiliza a Árvore Sintática Abstrata (AST) construída pelo *parser* como estrutura principal de entrada.

Essa etapa é implementada pela função recursiva `verificar_semantica()`, responsável por percorrer a AST e aplicar as verificações semânticas da linguagem. Durante esse percurso, são utilizados os mecanismos apresentados anteriormente, como a tabela de símbolos e o gerenciamento de escopos, permitindo validar declarações, referências a identificadores, compatibilidade de tipos e chamadas de função.

Além da detecção de erros, a análise semântica também realiza a inferência de tipos das expressões. O tipo calculado para cada nó é armazenado no campo `exp_type` da estrutura `ASTNode`, possibilitando o reaproveitamento dessas informações durante a geração de código intermediário.

Sempre que uma inconsistência é encontrada, o compilador emite uma mensagem contendo a descrição do erro e sua localização no código-fonte. A ocorrência de qualquer erro ativa a *flag* global `semantic_error`, impedindo a execução das etapas posteriores do compilador.

As próximas subseções apresentam em detalhes as verificações realizadas durante a análise semântica.

4.4.2 Verificação de Declarações

A validação de declarações é realizada durante o percurso da AST por meio dos nós `AST_VAR_DECL`, `AST_FUNC_DECL` e `AST_ID`. Essas verificações utilizam diretamente a tabela de símbolos apresentada anteriormente para registrar novos identificadores e validar referências realizadas ao longo do programa.

O trecho a seguir resume o fluxo adotado pelo compilador para tratar cada uma dessas construções durante a análise semântica.

```
1 switch (node->type) {
2
3 case AST_VAR_DECL:
4     add_symbol(...);
5     verificar_semantica(expressao);
6     break;
7
8 case AST_FUNC_DECL:
9     add_symbol(...);
10    registrar assinatura;
11    open_scope();
12    registrar parametros;
13    verificar_semantica(corpo);
14    close_scope();
```

```
15     break;
16
17 case AST_ID:
18     lookup_symbol(...);
19     reportar erro se não encontrado;
20     break;
21 }
```

No caso das declarações de variáveis (`AST_VAR_DECL`), cada identificador encontrado é inserido na tabela de símbolos do escopo corrente por meio da função `add_symbol()`. Como a inserção realiza previamente uma busca no escopo atual, o compilador é capaz de detectar redeclarações de variáveis dentro do mesmo bloco, emitindo uma mensagem de erro contendo a posição exata da ocorrência.

Além das declarações simples, o compilador também suporta declarações acompanhadas de inicialização. Nesses casos, após o registro do identificador na tabela de símbolos, a expressão utilizada na inicialização é analisada semanticamente para verificar sua compatibilidade com o tipo declarado da variável.

As declarações de função são tratadas pelos nós `AST_FUNC_DECL`. Inicialmente, o compilador verifica a validade do tipo de retorno e registra a função na tabela de símbolos global utilizando a categoria `func`. Em seguida, os tipos dos parâmetros são armazenados na estrutura do símbolo correspondente, permitindo que chamadas futuras sejam comparadas com a assinatura originalmente declarada.

Após o registro da função, um novo escopo é criado para representar seu corpo. Os parâmetros formais são inseridos na tabela de símbolos desse escopo com a categoria `param`, tornando-os visíveis durante a análise semântica da função.

Por fim, referências a identificadores são verificadas por meio dos nós `AST_ID`. Sempre que um identificador é encontrado, a função `lookup_symbol()` é utilizada para procurá-lo no escopo atual e, caso necessário, nos escopos ancestrais. Se nenhum símbolo correspondente for encontrado, o compilador reporta um erro de variável não declarada, impedindo a continuação correta da análise semântica.

4.4.3 Sistema de Tipos

O sistema de tipos do compilador é implementado durante o percurso da AST e tem como principal objetivo garantir a compatibilidade entre operações, atribuições, retornos de função e expressões utilizadas em estruturas de controle. Além da detecção de incompatibilidades, essa etapa também é responsável por inferir o tipo resultante de cada expressão e armazená-lo no campo `exp_type` do respectivo nó da árvore.

As regras de tipagem implementadas seguem os requisitos especificados para a linguagem e estão resumidas na Tabela 4.1.

Tabela 4.1: Regras de tipagem implementadas pelo compilador.

Construção	Operandos aceitos	Tipo resultante
Atribuição	int $\rightarrow$ int, int $\rightarrow$ float, float $\rightarrow$ float	Tipo da variável
+, -, *, /	int e float	Promoção implícita para float
%	Apenas int	int
<, <=, >, >=	int e float	int
==, !=	int e float	int
&&,	Expressões numéricas	int
!	Expressões numéricas	int
- unário	int e float	Mesmo tipo do operando
Condição de if/while	Expressões numéricas	-
return	Compatível com a função	Tipo da função

A Figura a seguir resume a lógica utilizada pelo compilador para determinar o tipo resultante das principais operações durante a análise semântica.

```

1  switch (node->type) {
2      case AST_ASSIGN:
3          verificar tipos da atribuição;
4          exp_type = tipo da variável;
5          break;
6
7      case AST_BINOP:
8          aplicar regras de promoção;
9          definir tipo resultante;
10         break;
11
12        case AST_UNOP:
13            definir tipo da expressão;
14            break;
15
16        case AST_IF:
17        case AST_WHILE:
18            validar condição numérica;
19            break;
20
21        case AST_RETURN:
22            verificar compatibilidade com o tipo da função;
23            break;
24    }

```

As atribuições são verificadas comparando os tipos da variável receptora e da expressão atribuída. O compilador permite conversões implícitas de int para float, porém impede a atribuição de valores float para variáveis declaradas como int, emitindo um erro semântico quando essa situação é identificada.

As expressões aritméticas seguem a política de promoção implícita de tipos. Sempre

que pelo menos um dos operandos possui tipo `float`, o resultado da operação também passa a ser do tipo `float`; caso contrário, o resultado permanece como `int`. A única exceção é o operador de módulo (%), cuja utilização é restrita exclusivamente a operandos inteiros.

Os operadores relacionais e lógicos produzem sempre resultados do tipo `int`, representando valores booleanos por meio de 0 e 1. Já o operador unário de negação lógica (!) também retorna obrigatoriamente um valor inteiro, enquanto o operador de menos unário preserva o tipo do operando.

Por fim, o compilador verifica se as condições utilizadas em comandos `if` e `while` correspondem a expressões numéricas e valida a compatibilidade entre o tipo declarado da função e o valor fornecido pelas instruções `return`. Durante todas essas verificações, o tipo inferido para cada expressão é armazenado no campo `exp_type` do respectivo nó da AST, permitindo sua reutilização pelas etapas posteriores de geração de código intermediário.

4.4.4 Verificação de Expressões

A verificação de expressões é realizada recursivamente durante o percurso da AST. Para cada operador encontrado, a função `verificar_semantica()` analisa inicialmente seus operandos e, somente após conhecer seus respectivos tipos, determina o tipo resultante da expressão.

Os nós correspondentes a identificadores e literais constituem os casos base dessa recursão. Literais inteiros e de ponto flutuante retornam diretamente seus respectivos tipos, enquanto identificadores consultam a tabela de símbolos para recuperar o tipo previamente registrado durante sua declaração.

Os operadores binários são tratados pelos nós `AST_BINOP`. Inicialmente, a função realiza chamadas recursivas para ambos os operandos, obtendo seus tipos. Em seguida, o operador armazenado no campo `lexema` determina qual regra de tipagem deve ser aplicada. O tipo resultante é então armazenado no campo `exp_type` do próprio nó, permitindo que expressões maiores reutilizem essa informação durante a continuação da análise.

A lógica implementada para operadores binários pode ser resumida pelo pseudocódigo apresentado a seguir.

```
1  t1 = verificar_semantica(child1);
2  t2 = verificar_semantica(child2);
3
4  if (op == "+" || op == "-" ||
5      op == "*" || op == "/") {
6
7      if (t1 == "float" || t2 == "float")
8          exp_type = "float";
9      else
10         exp_type = "int";
```

```
11 }
12
13 else if (op == "%") {
14     if (t1 == "float" || t2 == "float")
15         reportar_erro();
16
17     exp_type = "int";
18 }
19
20
21 else if (op é relacional ou lógico) {
22     exp_type = "int";
23 }
24
25 return exp_type;
```

Os operadores unários são processados de maneira semelhante. Inicialmente é realizada a análise semântica do operando e, em seguida, o tipo resultante é definido de acordo com o operador utilizado, conforme ilustrado a seguir.

```
1 t = verificar_semantica(child1);
2
3 if (op == "!")
4     exp_type = "int";
5 else
6     exp_type = t;
7
8 return exp_type;
```

Por fim, o tipo inferido de cada expressão é armazenado diretamente no campo `exp_type` da estrutura `ASTNode`. Dessa forma, a inferência de tipos é realizada apenas uma vez durante a análise semântica, permitindo que a etapa de geração de código intermediário reutilize essas informações sem a necessidade de percorrer novamente a árvore para determinar os tipos das expressões.

4.4.5 Verificação de Chamadas de Função

A validação de chamadas de função é baseada nas assinaturas registradas durante a análise das declarações de funções. Quando um nó `AST_FUNC_DECL` é processado, o compilador armazena na tabela de símbolos o tipo de retorno da função, a quantidade de parâmetros e o tipo correspondente de cada parâmetro. Essas informações constituem a assinatura da função e serão utilizadas posteriormente para validar todas as chamadas realizadas no programa.

Quando um nó `AST_FUNC_CALL` é encontrado durante o percurso da AST, a primeira etapa consiste em localizar o identificador correspondente na tabela de símbolos utilizando a função `lookup_symbol()`. Além da existência do identificador, o compilador

verifica se o símbolo recuperado pertence à categoria `func`, impedindo que variáveis sejam utilizadas como chamadas de função.

Após localizar a função, cada argumento informado na chamada é analisado recursivamente por meio da própria função `verificar_semantica()`, permitindo determinar seu tipo. Em seguida, o compilador compara os tipos obtidos com aqueles armazenados na assinatura da função. A implementação adota a mesma política de conversão implícita descrita na Subseção anterior, permitindo a promoção de `int` para `float`, mas rejeitando argumentos do tipo `float` quando o parâmetro correspondente espera um valor inteiro.

Além da compatibilidade de tipos, também é realizada a validação da quantidade de argumentos fornecidos. Ao término da análise, o número de argumentos encontrados é comparado com o número de parâmetros registrados na assinatura da função, sendo reportado um erro sempre que houver divergência.

A lógica empregada durante a validação de uma chamada de função pode ser resumida pelo pseudocódigo a seguir.

```
1  localizar_funcao();
2
3  if (funcao_inexistente)
4      reportar_erro();
5
6  para cada argumento {
7
8      tipo = verificar_semantica(argumento);
9
10     comparar(tipo, parametro_esperado);
11
12     if (tipos_incompativeis)
13         reportar_erro();
14 }
15
16 if (quantidade_argumentos != quantidade_parametros)
17     reportar_erro();
18
19 exp_type = tipo_de_retorno_da_funcao;
```

Após todas as verificações, o tipo de retorno armazenado na assinatura da função é copiado para o campo `exp_type` do nó correspondente à chamada. Dessa forma, chamadas de função podem participar normalmente de expressões maiores, reutilizando o mesmo mecanismo de inferência de tipos empregado para os demais nós da AST.

4.4.6 Relato de Erros Semânticos

Sempre que uma inconsistência é identificada durante a análise semântica, o compilador emite uma mensagem de erro padronizada iniciada pelo prefixo `[SEMANTIC ERROR]`. As mensagens descrevem explicitamente o problema encontrado, facilitando a identificação da causa do erro pelo usuário.

Uma característica importante da implementação é que todas as mensagens informam a posição exata da ocorrência no código-fonte. Para isso, a análise semântica utiliza os campos `line` e `column` armazenados nos nós da AST durante sua construção pelo analisador sintático. Sempre que possível, o erro é associado diretamente ao nó responsável pela inconsistência, como o identificador de uma variável, um argumento incompatível ou uma expressão com tipo inválido, tornando os diagnósticos mais precisos.

Ao detectar um erro, a função `verificar_semantica()` registra a ocorrência por meio da *flag* global `semantic_error`, mas não interrompe imediatamente a análise. Em vez disso, o percurso da AST continua normalmente, permitindo identificar múltiplos erros semânticos em uma única execução do compilador.

O fluxo geral empregado durante essa etapa pode ser resumido pelo pseudocódigo a seguir.

```
1 open_scope();
2
3 para cada nó da AST
4     verificar_semantica(no);
5
6 if (semantic_error)
7     encerrar_compilacao;
8
9 gerar_codigo_intermediario();
```

Após a conclusão da análise, a *flag* `semantic_error` é utilizada para decidir se a etapa de geração de código intermediário poderá ser executada. Caso pelo menos um erro tenha sido encontrado, o compilador informa que o programa foi rejeitado e encerra sua execução antes da geração do código intermediário. Dessa forma, preserva-se a arquitetura de múltiplas passagens adotada pelo projeto, garantindo que apenas programas semanticamente válidos avancem para as etapas posteriores da compilação.

4.5 Geração de Código Intermediário

4.5.1 Visão Geral da Geração de Código

Após a conclusão da análise semântica, o compilador inicia a etapa de geração de código intermediário. Assim como na fase anterior, todo o processamento é realizado por meio de um percurso recursivo sobre a Árvore Sintática Abstrata (AST), reutilizando as informações produzidas durante a verificação semântica, como os tipos inferidos das expressões armazenados no campo `exp_type`.

A geração do Código de Três Endereços (*Three Address Code* – TAC) é implementada pela função `gerar_tac()`, responsável por percorrer a árvore e emitir, em um arquivo texto, as instruções correspondentes a cada construção da linguagem.

De maneira análoga à análise semântica, a função utiliza um comando `switch` sobre o campo `type` da estrutura `ASTNode`. Cada categoria de nó possui uma estratégia própria de geração de código, permitindo tratar separadamente declarações, expressões, comandos de controle, chamadas de função e operações de entrada e saída.

O percurso é realizado de forma recursiva. Inicialmente são geradas as instruções correspondentes aos nós filhos e, em seguida, é emitido o código referente ao nó corrente. Essa estratégia garante que os operandos de uma expressão sejam sempre avaliados antes da operação que os utiliza, preservando corretamente as dependências entre as instruções produzidas.

A estrutura geral da função pode ser resumida pelo pseudocódigo apresentado a seguir.

```
1  gerar_tac(no):
2
3      se no == NULL
4          retorna
5
6      switch (no->type)
7
8          caso AST_VAR_DECL:
9              gerar código da declaração
10
11         caso AST_ASSIGN:
12             gerar código da expressão
13             emitir atribuição
14
15         caso AST_BINOP:
16             gerar código dos operandos
17             emitir operação
18
19         caso AST_IF:
20             gerar código condicional
21
22         caso AST_WHILE:
23             gerar código do laço
24
25         ...
26
27     fim switch
```

Durante esse percurso, cada nó de expressão produz um resultado associado ao campo `temp` da própria estrutura `ASTNode`. Esse campo armazena o nome do temporário que contém o valor calculado pela expressão, permitindo que instruções subsequentes reutilizem diretamente o resultado anteriormente produzido.

Além da emissão das instruções propriamente ditas, a geração de código também utiliza informações inferidas durante a análise semântica para inserir conversões explícitas de tipo quando necessárias. Dessa forma, o código intermediário produzido repre-

senta fielmente o comportamento definido pelas regras de tipagem implementadas pelo compilador.

As próximas subseções apresentam detalhadamente a estrutura do Código de Três Endereços adotado, bem como as estratégias utilizadas para geração de expressões, estruturas de controle, chamadas de função e gerenciamento de temporários e rótulos.

4.5.2 Estrutura do Código de Três Endereços

O código intermediário gerado pelo compilador utiliza uma representação baseada em Código de Três Endereços (*Three Address Code* – TAC). Nessa forma intermediária, cada instrução executa uma única operação elementar, facilitando etapas posteriores de tradução para código de máquina ou otimizações.

A implementação adota uma representação textual simples, na qual cada instrução é escrita em uma linha do arquivo `output.tac`. Expressões mais complexas são decompostas em operações sucessivas, utilizando variáveis temporárias para armazenar resultados intermediários.

As principais instruções utilizadas pelo compilador são apresentadas na Tabela 4.2.

Tabela 4.2: Conjunto de instruções do Código de Três Endereços implementado.

Instrução	Descrição	Exemplo
Atribuição	Copia um valor para uma variável ou temporário.	<code>x = t1</code>
Operação binária	Executa uma operação entre dois operandos.	<code>t3 = t1 + t2</code>
Operação unária	Aplica um operador sobre um único operando.	<code>t2 = - t1</code>
Conversão de tipo	Realiza uma promoção explícita entre tipos.	<code>t1 = (float) x</code>
Desvio incondicional	Transfere o fluxo para um rótulo.	<code>goto L3</code>
Desvio condicional	Seleciona o próximo bloco conforme o resultado da condição.	<code>if t1 goto L1</code>
Rótulo	Marca um ponto de destino no código.	<code>L2:</code>
Passagem de parâmetro	Empilha um argumento antes da chamada.	<code>param x</code>
Recepção de parâmetro	Recupera um parâmetro no início da função.	<code>pop_param a</code>
Chamada de função	Invoca uma função e armazena seu retorno.	<code>t1 = call soma, 2</code>
Retorno	Finaliza uma função retornando um valor.	<code>return t1</code>
Entrada	Lê um valor para uma variável.	<code>read x</code>
Saída	Escreve o valor de uma expressão.	<code>print t1</code>

Essa representação foi definida de forma a manter cada instrução suficientemente simples para representar apenas uma operação por vez. Sempre que uma expressão envolve múltiplos operadores, o compilador a decompõe em várias instruções elementares, utilizando temporários para preservar os resultados intermediários.

Outro aspecto importante da implementação é que conversões implícitas identificadas durante a análise semântica são representadas explicitamente no código intermediário. Assim, promoções de `int` para `float`, por exemplo, tornam-se instruções

específicas de conversão, preservando de forma explícita a semântica do programa para as etapas posteriores do compilador.

4.5.3 Temporários e Rótulos

A geração do Código de Três Endereços utiliza identificadores temporários e rótulos para representar resultados intermediários de expressões e controlar o fluxo de execução do programa.

Os temporários são utilizados sempre que uma expressão produz um valor que será reutilizado posteriormente. Cada novo temporário recebe um identificador único no formato `t1`, `t2`, `t3`, ..., gerado por meio de um contador global incrementado a cada solicitação.

A lógica empregada pode ser resumida pelo pseudocódigo apresentado a seguir.

```
1 new_temp():
2
3     cria nome "t" + temp_counter
4
5     incrementa temp_counter
6
7     retorna o novo nome
```

Após sua criação, o nome do temporário é armazenado no campo `temp` do respectivo nó da AST. Dessa forma, quando um nó pai necessita utilizar o resultado produzido por uma subexpressão, basta consultar esse campo, sem necessidade de recalculá-lo ou localizá-lo novamente.

Os rótulos desempenham papel semelhante no controle do fluxo de execução. Cada rótulo identifica um ponto específico do código intermediário e é utilizado por instruções de desvio condicional e incondicional, implementando estruturas como `if`, `if-else`, `while` e a avaliação em curto-circuito dos operadores lógicos.

Assim como os temporários, os rótulos são gerados automaticamente por meio de um contador global, produzindo identificadores sequenciais no formato `L1`, `L2`, `L3`,

```
1 new_label():
2
3     cria nome "L" + label_counter
4
5     incrementa label_counter
6
7     retorna o novo nome
```

O exemplo a seguir ilustra a utilização conjunta de temporários e rótulos durante a geração de código intermediário.

```
1  t1 = a + b
2  t2 = t1 * c
3
4  if t2 goto L1
5  goto L2
6
7  L1:
8  print t2
9
10 L2:
```

Essa estratégia garante que cada temporário e cada rótulo possuam identificadores únicos ao longo de toda a compilação, simplificando tanto a geração do código intermediário quanto sua posterior tradução para uma linguagem de mais baixo nível.

4.5.4 Geração de Código para Expressões

A geração de código para expressões é realizada de forma recursiva, acompanhando a própria estrutura da AST. Assim como ocorre na análise semântica, os nós folha são processados primeiro e seus resultados são propagados para os nós superiores, permitindo construir o código intermediário seguindo a ordem natural de avaliação da expressão.

Os casos base da recursão correspondem aos nós `AST_INT`, `AST_FLOAT` e `AST_ID`. Nesses casos, nenhuma instrução TAC é emitida. Apenas o campo `temp` do nó é preenchido com a representação do valor constante ou com o nome do identificador, possibilitando que esses valores sejam reutilizados pelas expressões que os utilizam como operandos.

As atribuições são tratadas pelos nós `AST_ASSIGN`. Inicialmente, a expressão do lado direito é completamente avaliada por meio de uma chamada recursiva a `gerar_tac()`. Em seguida, caso a análise semântica tenha determinado que a variável destino possui tipo `float` e a expressão produz um valor inteiro, é gerada explicitamente uma instrução de conversão antes da atribuição. Por fim, é emitida a instrução responsável por armazenar o resultado na variável correspondente.

Os operadores binários são implementados pelos nós `AST_BINOP`. Após gerar o código de ambos os operandos, o compilador consulta os tipos previamente inferidos durante a análise semântica por meio do campo `exp_type`. Caso seja necessária alguma promoção implícita de tipos, uma instrução de conversão é inserida antes da operação propriamente dita. Em seguida, um novo temporário é criado para armazenar o resultado da expressão.

A lógica empregada pode ser resumida pelo pseudocódigo a seguir.

```
1  gerar_tac(child1);
2  gerar_tac(child2);
3
```

```
4 op1 = child1->temp;
5 op2 = child2->temp;
6
7 if (child1->exp_type == "int" &&
8     child2->exp_type == "float") {
9
10     cast = new_temp();
11     emitir(cast = (float) op1);
12     op1 = cast;
13 }
14
15 else if (child1->exp_type == "float" &&
16         child2->exp_type == "int") {
17
18     cast = new_temp();
19     emitir(cast = (float) op2);
20     op2 = cast;
21 }
22
23 temp = new_temp();
24 emitir(temp = op1 operador op2);
```

Os operadores unários seguem uma estratégia semelhante. Inicialmente, o operando é processado recursivamente e, em seguida, um novo temporário é criado para armazenar o resultado da operação unária, preservando o modelo de código de três endereços adotado pelo compilador.

Uma característica importante dessa implementação é o reaproveitamento das informações produzidas pela análise semântica. Em vez de recalcular tipos durante a geração do TAC, o compilador utiliza diretamente o conteúdo armazenado no campo `exp_type` de cada nó da AST para decidir quando instruções de conversão devem ser inseridas, mantendo uma clara separação entre as responsabilidades das duas etapas do compilador.

4.5.5 Geração de Código para Estruturas de Controle

A geração de código para estruturas de controle é baseada na criação de rótulos (*labels*) e instruções de desvio (*goto*), permitindo representar o fluxo de execução por meio de Código de Três Endereços.

Uma decisão de projeto adotada foi separar a geração de expressões da geração de condições. Enquanto a função `gerar_tac()` é responsável por produzir o código das expressões e comandos em geral, a função `gerar_condicao()` trata exclusivamente da tradução de expressões booleanas utilizadas em comandos condicionais e de repetição.

Essa separação permite que estruturas como `if` e `while` deleguem toda a geração da condição para uma rotina especializada, tornando a implementação mais modular e facilitando a aplicação de otimizações, como a avaliação em curto-circuito dos operadores lógicos.

A organização geral empregada para a geração de um comando condicional pode ser resumida pelo pseudocódigo apresentado a seguir.

```
1  l_true  = new_label();
2  l_false = new_label();
3
4  gerar_condicao(condicao, l_true, l_false);
5
6  l_true:
7      gerar_tac(bloco_if);
8
9  if (existe_else) {
10     goto l_end;
11
12     l_false:
13         gerar_tac(bloco_else);
14
15     l_end:
16 }
17 else {
18     l_false:
19 }
```

A geração de laços de repetição segue a mesma estratégia, acrescentando um rótulo correspondente ao início do laço, para o qual o fluxo retorna ao término de cada iteração.

```
1  l_start = new_label();
2  l_true  = new_label();
3  l_false = new_label();
4
5  l_start:
6      gerar_condicao(condicao, l_true, l_false);
7
8  l_true:
9      gerar_tac(corpo);
10     goto l_start;
11
12 l_false:
```

A função `gerar_condicao()` implementa a tradução das expressões lógicas utilizando avaliação em curto-circuito (*short-circuit evaluation*). Para operadores lógicos (`&&`, `||` e `!`), a função gera novos rótulos intermediários e decide dinamicamente se a avaliação da segunda subexpressão é realmente necessária.

No caso do operador `||`, caso a primeira condição seja verdadeira, o fluxo é imediatamente desviado para o rótulo de sucesso, dispensando a avaliação da segunda expressão. De forma análoga, para o operador `&&`, caso a primeira condição seja falsa, o compilador desvia diretamente para o rótulo de falha, evitando cálculos desnecessários.

Já o operador de negação lógica (!) é implementado simplesmente invertendo os rótulos de verdadeiro e falso recebidos pela função.

Quando a condição corresponde a uma comparação relacional, ambos os operandos são inicialmente traduzidos para TAC por meio da função `gerar_tac()`. Em seguida, caso seja necessário, são inseridas conversões implícitas entre `int` e `float`, reutilizando as informações de tipo produzidas durante a análise semântica. Finalmente, são emitidas as instruções de desvio condicional e incondicional que controlam o fluxo da execução.

Para condições formadas apenas por uma variável ou expressão numérica, como em `if(x)` ou `while(1)`, a implementação gera inicialmente o código correspondente à expressão `e`, em seguida, realiza uma comparação implícita contra zero, preservando o comportamento esperado da linguagem.

4.5.6 Geração de Código para Funções

A geração de código para funções contempla a tradução de declarações de função, chamadas e instruções de retorno. A implementação adota um modelo baseado em passagem explícita de parâmetros, no qual o envio dos argumentos e sua recuperação dentro da função são representados por instruções específicas do Código de Três Endereços.

Quando um nó `AST_FUNC_DECL` é processado, o compilador inicialmente emite um rótulo correspondente ao nome da função. Em seguida, para cada parâmetro formal declarado, é gerada uma instrução `pop_param`, responsável por recuperar o argumento previamente enviado pela chamada. Após essa etapa, a geração de código prossegue normalmente sobre o corpo da função.

A lógica empregada pode ser resumida pelo pseudocódigo apresentado a seguir.

```
1 funcao:
2
3 para cada parametro
4     pop_param parametro
5
6 gerar_tac(corpo_da_funcao)
```

As chamadas de função são tratadas pelos nós `AST_FUNC_CALL`. Inicialmente, cada argumento é traduzido recursivamente por meio da função `gerar_tac()`, garantindo que seu valor esteja disponível em um temporário. Em seguida, os argumentos são enviados utilizando instruções `param`. Ao término desse processo, o compilador gera uma instrução `call`, informando o nome da função e a quantidade de argumentos enviados.

O valor retornado pela chamada é armazenado em um novo temporário, permitindo que chamadas de função sejam utilizadas diretamente como operandos de expressões maiores.

A estratégia utilizada pode ser resumida pelo pseudocódigo a seguir.

```
1 para cada argumento
```

```
2      gerar_tac(argumento)
3      param argumento.temp
4
5  temp = new_temp()
6  temp = call funcao, quantidade_argumentos
```

As instruções de retorno são representadas pelos nós `AST_RETURN`. Quando existe uma expressão associada ao comando `return`, seu código é gerado previamente e o valor produzido é utilizado na instrução de retorno. Caso contrário, é emitida apenas a instrução `return`, indicando um retorno sem valor.

Essa organização mantém a geração de código independente da etapa de análise semântica. Todas as verificações relacionadas à existência das funções, compatibilidade entre parâmetros, tipos de retorno e quantidade de argumentos já foram realizadas anteriormente, permitindo que a geração de TAC concentre-se exclusivamente na tradução da AST para instruções intermediárias.

4.6 Decisões de Projeto

4.6.1 Arquitetura em Fases Estritas (Pipeline)

Uma das principais decisões arquiteturais do compilador foi a adoção de um modelo de múltiplas passagens (*multi-pass*). Nesse modelo, a primeira passagem corresponde à análise léxica e sintática, executadas de forma integrada durante a construção da AST. Após a conclusão dessa etapa, são realizadas, em passagens independentes, a análise semântica e, por fim, a geração de código intermediário. Cada fase somente é iniciada após a validação completa da etapa anterior.

Essa organização pode ser observada diretamente na função principal do compilador, responsável por coordenar o fluxo de execução. Inicialmente é realizada a análise léxica e sintática por meio da chamada à função `yyparse()`. Somente quando nenhuma inconsistência é detectada a execução prossegue para a análise semântica, que percorre a AST previamente construída. Da mesma forma, a geração do código intermediário é iniciada apenas quando toda a verificação semântica é concluída com sucesso.

O fluxo geral adotado pode ser resumido conforme o pseudocódigo apresentado a seguir.

```
1  executar analise lexica e sintatica;
2
3  if (existem erros)
4      abortar compilacao;
5
6  executar analise semantica;
7
8  if (existem erros)
9      abortar compilacao;
```

10

11 `gerar codigo intermediario;`

Observa-se que a primeira passagem engloba simultaneamente as análises léxica e sintática, enquanto as duas passagens seguintes são dedicadas, respectivamente, à análise semântica e à geração do código intermediário.

Essa arquitetura proporciona uma clara separação de responsabilidades entre as fases do compilador. Cada etapa trabalha sobre os resultados produzidos pela anterior, sem realizar verificações pertencentes às demais fases, tornando a implementação mais modular, organizada e de fácil manutenção.

Além disso, o projeto adota a estratégia conhecida como *fail-fast*. Para isso, são utilizadas as variáveis globais `lexical_error`, `syntax_error` e `semantic_error`, que interrompem imediatamente o processo de compilação quando algum erro é encontrado. Com essa decisão, evita-se que fases posteriores operem sobre estruturas inconsistentes, impedindo a geração de código intermediário inválido e simplificando o tratamento de erros ao longo da implementação.

4.6.2 Projeto da Árvore Sintática Abstrata

Uma das principais decisões de projeto adotadas foi desacoplar completamente a construção da Árvore Sintática Abstrata (AST) das etapas posteriores do compilador. Em vez de realizar verificações semânticas ou emitir código intermediário diretamente durante as ações da gramática, a análise sintática possui como única responsabilidade construir uma representação completa do programa em memória.

Essa abordagem caracteriza uma arquitetura de múltiplas passagens (*multi-pass*), na qual cada fase do compilador percorre a AST de maneira independente. Após sua construção, a árvore é reutilizada pela análise semântica para validação das regras da linguagem e, posteriormente, pela geração de código intermediário para emissão das instruções TAC. Dessa forma, cada etapa atua sobre uma estrutura já consolidada, sem depender diretamente da execução das fases anteriores.

Outra decisão importante foi utilizar uma única estrutura genérica (`ASTNode`) para representar todos os elementos da linguagem. Em vez de criar estruturas específicas para cada construção sintática, optou-se por uma representação uniforme baseada em um identificador de tipo, ponteiros para filhos e ponteiros para irmãos. Essa solução reduz significativamente a quantidade de estruturas necessárias e permite que algoritmos de percurso, como a análise semântica e a geração de código, sejam implementados de forma genérica por meio de uma única função recursiva.

A utilização do ponteiro `next` para encadear nós do mesmo nível também simplifica a representação de listas de declarações, parâmetros, argumentos de chamadas e sequências de comandos, eliminando a necessidade de criar nós intermediários apenas para representar coleções de elementos.

Por fim, cada nó da AST armazena, desde sua criação, informações de linha e coluna

correspondentes ao trecho original do código-fonte. Essa decisão permite que etapas posteriores, especialmente a análise semântica, produzam mensagens de erro precisas sem depender de informações provenientes do analisador léxico ou sintático, tornando o processo de diagnóstico mais simples e desacoplado das fases anteriores.

4.6.3 Análise Semântica e Sistema de Tipos

Durante o desenvolvimento da análise semântica, optou-se por adotar um modelo de escopo estático baseado em uma pilha de tabelas de símbolos. Sempre que um novo bloco ou função é iniciado, uma nova tabela de símbolos é criada e associada ao escopo corrente. Ao término do bloco, o compilador retorna automaticamente ao escopo pai. Essa estratégia reproduz o comportamento tradicional das linguagens imperativas e permite implementar naturalmente regras como visibilidade de identificadores e sombreamento de variáveis.

Outra decisão importante refere-se ao sistema de tipos adotado pelo compilador. Buscando aproximar o comportamento da linguagem ao padrão encontrado em linguagens como C, foi implementada a promoção implícita apenas no sentido de `int` para `float`. Dessa forma, operações envolvendo ambos os tipos produzem resultados em ponto flutuante, enquanto atribuições e retornos que poderiam ocasionar perda de precisão, como converter um `float` para `int`, são tratados como erros semânticos.

Essa mesma política é refletida durante a geração de código intermediário, que insere instruções explícitas de conversão sempre que uma promoção implícita é necessária. Dessa forma, as decisões tomadas pela análise semântica permanecem preservadas nas etapas posteriores do compilador.

Também foi adotada a convenção de que operadores relacionais e operadores lógicos produzem sempre resultados do tipo `int`, utilizando os valores 0 e 1 para representar falso e verdadeiro, respectivamente. Embora internamente a linguagem não possua um tipo booleano específico, essa decisão simplifica tanto a análise semântica quanto a geração de código, além de manter compatibilidade com o comportamento clássico da linguagem C.

4.6.4 Geração de Código Intermediário

A geração de código intermediário foi projetada para produzir uma representação simples, porém suficientemente expressiva para preservar o comportamento do programa fonte. Para isso, adotou-se como representação intermediária o Código de Três Endereços (TAC), amplamente utilizado em compiladores por facilitar etapas posteriores de otimização e tradução para código de máquina.

Uma das principais decisões adotadas foi implementar avaliação em curto-circuito (*short-circuit*) para operadores lógicos. Em vez de avaliar todos os operandos de uma expressão composta, o compilador gera sequências de desvios condicionais que interrompem a avaliação assim que o resultado lógico é determinado. Essa estratégia reduz a

quantidade de instruções geradas e reproduz o comportamento esperado em linguagens como C.

Outra decisão importante foi tornar explícitas no código intermediário todas as conversões implícitas de tipos identificadas durante a análise semântica. Sempre que uma promoção de `int` para `float` é necessária, o compilador gera uma instrução de conversão antes da operação correspondente. Dessa forma, o TAC preserva completamente as decisões de tipagem tomadas na etapa anterior, tornando explícitas operações que, na linguagem fonte, ocorrem de maneira implícita.

Por fim, optou-se por representar chamadas de função utilizando um protocolo baseado em passagem explícita de parâmetros. Antes da instrução de chamada, cada argumento é emitido por meio de uma instrução `param`, enquanto a função receptora recupera seus parâmetros utilizando instruções `pop_param`. Embora simplificada, essa estratégia aproxima o código intermediário do modelo clássico baseado em pilha utilizado por diversas arquiteturas, além de desacoplar a geração dos argumentos da execução da chamada propriamente dita.

4.7 Dificuldades Encontradas

Durante o desenvolvimento do compilador, algumas etapas exigiram um cuidado maior de implementação por envolverem a integração entre diferentes fases do processo de compilação. Esta seção apresenta as principais dificuldades enfrentadas durante o projeto e as soluções adotadas para superá-las.

4.7.1 Localização Precisa de Erros Semânticos

Uma das primeiras dificuldades encontradas foi produzir mensagens de erro semântico capazes de indicar não apenas a linha, mas também a coluna exata em que a inconsistência ocorreu. Durante a construção da AST, diversos nós são criados a partir da redução de produções da gramática, fazendo com que a posição original do elemento analisado pudesse ser perdida caso essa informação não fosse preservada.

A solução adotada consistiu em incorporar os campos `line` e `column` diretamente à estrutura `ASTNode`. Essas informações são preenchidas no momento da criação dos nós utilizando os dados fornecidos pelo analisador léxico ou herdadas do primeiro filho durante as reduções da gramática. Como consequência, todas as etapas posteriores do compilador passam a ter acesso à posição exata de cada elemento da árvore, permitindo que os erros semânticos sejam reportados com maior precisão.

4.7.2 Implementação da Promoção de Tipos

Outra dificuldade significativa foi implementar corretamente a promoção automática de tipos entre valores inteiros e de ponto flutuante. Em uma implementação inicial, as verificações exigiam que ambos os operandos possuísem exatamente o mesmo tipo, fazendo com que expressões válidas fossem rejeitadas durante a análise semântica.

A solução consistiu em incorporar um mecanismo de inferência e promoção de tipos baseado nas regras da linguagem. Durante a análise semântica, cada expressão passa a armazenar seu tipo inferido por meio do campo `exp_type`. Posteriormente, durante a geração de código intermediário, essas informações permitem inserir instruções explícitas de conversão apenas quando necessário, preservando o comportamento esperado da linguagem e evitando perdas de informação.

4.7.3 Gerenciamento de Escopos

O gerenciamento de escopos também representou um desafio importante durante a implementação da análise semântica. Como a linguagem permite blocos aninhados e declarações locais, foi necessário garantir que cada identificador permanecesse visível apenas dentro do escopo em que foi declarado, respeitando ainda o mecanismo de sombreamento entre escopos.

Para resolver esse problema, foi adotada uma estrutura hierárquica de tabelas de símbolos, na qual cada escopo mantém uma referência para seu escopo pai. As funções responsáveis pela abertura e fechamento de escopos passaram a controlar automaticamente a tabela ativa durante o percurso da AST, enquanto as operações de busca percorrem essa hierarquia até localizar um identificador ou concluir que ele não foi declarado. Essa abordagem permitiu implementar corretamente a visibilidade de símbolos, detectar redeclarações em um mesmo escopo e validar referências a variáveis e funções de maneira consistente.

4.8 Saída do Analisador

Nesta seção são apresentados dois exemplos de execução do compilador desenvolvido. O primeiro corresponde a um programa semanticamente correto, permitindo observar todo o fluxo de compilação até a geração do Código de Três Endereços (TAC). O segundo contém um erro semântico proposital, ilustrando o mecanismo de detecção de erros e a interrupção da compilação antes da geração do código intermediário.

4.8.1 Caso de Sucesso

Para validar o funcionamento do compilador foi utilizado o programa apresentado na Listagem 2. Esse exemplo exercita a maior parte dos recursos implementados pela linguagem, incluindo declaração de funções, passagem de parâmetros, operações aritméticas, estruturas condicionais, laços de repetição, chamadas às funções `read` e `print`, além de conversões implícitas entre tipos numéricos.

```
1  int soma(int a, int b) {
2      int resultado;
3      resultado = a + b;
4      return resultado;
5  }
6  /* Isso é um comentário multilinha
7  para testar algo */
8
9  float main( ){
10     int x, w;
11     int y;
12     float z;
13
14     read(x);
15     read(y);
16
17     z = soma(x, y);
18
19     return z;
20 }
```

Listagem 2: Programa utilizado no caso de sucesso.

Durante a execução, o compilador realiza inicialmente as análises léxica e sintática, exibindo os tokens reconhecidos juntamente com suas posições no código-fonte. Em seguida, a análise semântica percorre a AST, valida todas as regras da linguagem e constrói as tabelas de símbolos correspondentes aos diferentes escopos identificados.

Como nenhuma inconsistência é encontrada, o programa é aceito e o compilador prossegue para a geração do Código de Três Endereços, produzindo automaticamente o arquivo `output.tac`.

```

1 =====
2 1. STARTING LEXICAL AND SYNTACTIC ANALYSIS
3 =====
4 KW_INT      | int      | 1      | 1
5 ID          | soma     | 1      | 5
6 LPAREN      | (        | 1      | 9
7 .
8 .
9 .
10 SEMCOL     | ;        | 19     | 13
11 RBRACE     | }        | 20     | 1
12
13 =====
14 2. STARTING SEMANTIC ANALYSIS
15 =====
16 Semantic analysis passed successfully.
17
18 =====
19                               FINAL SYMBOL TABLES DUMP
20 =====
21
22 =====
23          TABLE ID: 0 | SCOPE LEVEL: 0 | PARENT ID: NULL (GLOBAL)
24 =====
25 NAME          | TYPE      | CATEGORY | NUM_PARAMS | PARAM_TYPES | NEXT SYMBOL
26 -----
27 main          | float     | func     | 0          | -           | soma
28 soma          | int       | func     | 2          | int, int    | NULL
29 .
30 .
31 .
32 =====
33 PROGRAM ACCEPTED. Intermediate code generated in 'output.tac'.
34 =====

```

Listagem 3: Trecho da saída produzida durante a compilação.

O código intermediário gerado preserva toda a estrutura lógica do programa, incluindo operações aritméticas, chamadas de função, conversões de tipos e estruturas de controle.

```
1 soma:
2 pop_param a
3 pop_param b
4 t1 = a + b
5 resultado = t1
6 return resultado
7
8 main:
9 read x
10 read y
11 param x
12 param y
13 t2 = call soma, 2
14 t3 = (float) t2
15 z = t3
16 return z
```

Listagem 4: Trecho do Código de Três Endereços gerado.

4.8.2 Caso de Erro Semântico

Também foi elaborado um segundo programa contendo uma inconsistência semântica proposital para validar o mecanismo de detecção de erros implementado pelo compilador. Nesse exemplo, o operador de resto da divisão (%) é aplicado sobre uma variável do tipo float, operação que não é permitida pela linguagem implementada.

```
1 int soma(int a, int b) {
2     int resultado;
3     resultado = a + b;
4     return resultado;
5 }
6 /* Isso é um comentário multilinha
7 para testar algo */
8
9 float main() {
10     int x, w;
11     int y;
12     float z;
13
14     read(x);
15     read(y);
16
17     z = soma(x, y);
18     z = z % x;
19
20     return z;
21 }
```

Listagem 5: Programa utilizado para validação de erros semânticos.

Durante a análise semântica, o compilador identifica a incompatibilidade de tipos e emite uma mensagem contendo a descrição do erro juntamente com sua linha e coluna no código-fonte. A ocorrência desse erro ativa a *flag* global `semantic_error`, interrompendo imediatamente o processo de compilação.

Como consequência, a etapa de geração do Código de Três Endereços não é executada e nenhum arquivo intermediário é produzido, garantindo que apenas programas semanticamente válidos possam prosseguir para as etapas posteriores do compilador.

```

1  =====
2  1. STARTING LEXICAL AND SYNTACTIC ANALYSIS
3  =====
4  KW_INT      | int      | 1      | 1
5  ID          | soma    | 1      | 5
6  LPAREN      | (       | 1      | 9
7  .
8  .
9  .
10 =====
11 2. STARTING SEMANTIC ANALYSIS
12 =====
13 [SEMANTIC ERROR] Operator '%' only accepts int at Line 18, Column 9.
14
15 PROGRAM REJECTED DUE TO SEMANTIC ERRORS

```

Listagem 6: Mensagem produzida pelo compilador ao detectar um erro semântico.

4.9 Conclusão

O presente trabalho apresentou o desenvolvimento de um compilador para uma linguagem de programação simplificada, contemplando as etapas de análise léxica, análise sintática, análise semântica e geração de código intermediário. Ao longo do projeto foram implementadas as principais estruturas presentes em compiladores modernos, incluindo a construção de uma Árvore Sintática Abstrata (AST), gerenciamento de escopos por meio de tabelas de símbolos hierárquicas, verificação de tipos e geração de Código de Três Endereços (TAC).

Os resultados obtidos demonstram que o compilador é capaz de reconhecer corretamente programas válidos, detectar inconsistências semânticas com mensagens de erro precisas e produzir uma representação intermediária compatível com a estrutura do programa de entrada. A utilização de uma arquitetura em múltiplas passagens mostrou-se adequada para manter uma separação clara entre as diferentes fases da compilação, tornando a implementação mais organizada, modular e de fácil manutenção.

Além de atender aos requisitos propostos, o desenvolvimento do projeto proporcionou uma compreensão prática dos principais conceitos estudados na disciplina de compiladores, evidenciando como técnicas clássicas de análise e tradução de programas podem ser integradas em uma única ferramenta funcional.

Como trabalhos futuros, o compilador pode ser estendido com o suporte a novos recursos da linguagem, como vetores, diferentes tipos de dados, otimizações sobre o código intermediário e geração de código para uma arquitetura alvo específica, aproximando ainda mais sua implementação da estrutura adotada por compiladores utilizados em ambientes reais.

Considerações Finais

O desenvolvimento deste projeto permitiu consolidar, de forma prática, os principais conceitos envolvidos na construção de compiladores. Ao longo do trabalho foram implementadas todas as etapas propostas, desde a análise léxica até a geração de código intermediário, resultando em um compilador funcional capaz de reconhecer programas escritos na linguagem definida, identificar erros sintáticos e semânticos e produzir uma representação intermediária consistente para programas válidos.

Além do atendimento aos requisitos estabelecidos, o projeto evidenciou a importância de decisões arquiteturais, como a adoção de uma estrutura em múltiplas passagens, a utilização de uma Árvore Sintática Abstrata como estrutura central de representação e o gerenciamento hierárquico de escopos por meio de tabelas de símbolos. Essas escolhas contribuíram para uma implementação modular, organizada e de fácil manutenção, além de aproximarem o compilador desenvolvido da arquitetura empregada por compiladores reais.

Por fim, o projeto proporcionou uma compreensão mais aprofundada sobre o funcionamento interno de compiladores, demonstrando como as diferentes fases do processo de compilação se integram para transformar um programa fonte em uma representação intermediária semanticamente válida. A experiência adquirida durante sua implementação constitui uma base sólida para estudos futuros envolvendo otimização de código, geração de código objeto e outras técnicas avançadas da área de compiladores.